Antigravity Operations Protocol

Token Efficiency & Strict Error-Validation Framework

1. Token & Quota Management Strategy

To operate continuously within daily limits without wasting quotas, agents must implement strict token efficiency guidelines. Antigravity quotas refresh every ~5 hours on premium tiers, meaning utilization must be paced and optimized.

Configuration Directives

- **Overage Protection:** Enforce the `AI Credit Overages` setting to **Never**. This ensures agents will pause and wait for the baseline 5-hour quota refresh rather than silently bleeding paid credits.
- **Thinking Levels:** By default, cap the reasoning depth via `GenerationConfig` to `MINIMAL` or `LOW` for routine tasks.
- **Context Compaction:** Utilize Antigravity's automatic context compaction (triggered at ~135k tokens). Implement *Transform* lifecycle hooks to aggressively truncate large log files or unnecessary payload data before it enters the model context.

Zero-Waste Scheduling

Leverage background execution to maximize daily throughput. Implement the smart quota-reset detection (Zero-Waste Mode). By synchronizing operations with the ~5-hour reset windows, agents can execute bulk data processing just before a reset, ensuring no quota is left on the table.

2. Post-Task Validation & Error Checking Session

Executing code blindly wastes both time and context window tokens on subsequent debugging turns. Agents are required to initiate an autonomous error-checking session immediately following implementation, utilizing the built-in `code_execution` tool.

Strict Validation Protocols

Before producing the final *Walkthrough* artifact, the agent must run the following validation suite against the active codebase:

- **Python Syntactic & Structural Checks:** Agents must execute `python -m py_compile` on all modified files. Furthermore, strict formatting checks must be enforced to validate precise indentation alignment, catching nested block syntax errors before runtime.
- **C++ & Kotlin Compilation:** For algorithmic implementations, agents must trigger local builds. Code must compile cleanly under `g++ -O2 -Wall` for C++ and the standard `kotlinc` compiler for Kotlin to ensure strict type safety and syntax adherence.
- **Test Case Execution:** Sample test cases must be rigorously applied. Agents must isolate edge cases, specifically validating initial parameters and array bounds, ensuring output precisely matches expected constraints.

> **Mandatory Hook:** Implement a `Decide` lifecycle hook that prevents the agent from closing a task or generating a Walkthrough if the error-checking compilation fails. The agent must loop back and fix syntax or logic errors immediately.